`pttai`: Build-Time Dataflow Validation for LLM Pipeline Graphs

Teerat Pipitcharulerd
Independent Researcher
teerat.pipitcharulerd@gmail.com

Abstract

LLM agent pipelines are graphs of model calls, tools, and branches over a shared state. A miswired one—a node that reads a state key nothing upstream produced, an unwired branch, a concurrent write to a key with no reducer—does not fail when you build it. It fails *after* the model has already run, once execution reaches the broken node, having burned tokens, latency, and (with tools) real side effects on a run that was doomed at construction. We present `pttai`, a thin declarative layer in which each node declares the state keys it `reads` and `writes`, wired with a > operator; it compiles to a native LangGraph (LangChain, 2024) `StateGraph` and, *before* compiling, runs a build-time availability (dataflow) analysis over the graph. The analysis is a MAY/MUST fixpoint over the graph's acyclic condensation, in the tradition of reaching-definitions and definite-assignment analysis (Kildall, 1973; Aho et al., 2006; Gosling et al., 2014): hard errors are raised only from the MAY (some-path) set, so it does not reject a pipeline that would have run—we observe **0** false positives on 19 valid pipelines, which we report as a measurement, not a guarantee. On a controlled 17-item bug suite `pttai` fails the build on all 17; on the 12-item subset that raw LangGraph does not also catch at build, LangGraph surfaces every case only at run time. Our primary, non-circular evaluation applies the same validator to pipelines a small OpenAI model (`gpt-5.4-nano`) generates from NLP task specs—a static guardrail for AI-written agent code (numbers pending a keyed generation run; §4). The demonstration is an interactive playground that paints the offending node red at build time, before any model call.

1 Introduction

Language-model pipelines have become graphs. A retrieval-augmented QA system retrieves passages, then answers grounded in them; an extract-then-summarize pipeline pulls typed fields from a ticket, then writes a one-line triage summary from those fields; a document-triage pipeline classifies an incoming message and routes it to a specialized handler. Frameworks such as LangGraph (LangChain, 2024) express these as explicit state-machine graphs: nodes read and write a shared state object, edges (some conditional) move control between them.

These graphs fail differently from ordinary programs. In a shared-state graph, a node that reads `state["summary"]` before any node has written `summary` does not raise when the graph is built—it raises a `KeyError` at run time, once control reaches that node. By then the pipeline has already made model calls: tokens spent, latency incurred, and, if earlier nodes called tools, side effects committed, all on a run that was structurally broken before it started. The same is true of an unwired conditional branch (a routing key that resolves to nowhere) and of two parallel branches writing the same non-reduced state key (a runtime `InvalidUpdateError`). These are recurring, publicly reported failures against raw LangGraph (§4.3).

The root cause is that the graph builder checks *structure* but not *dataflow*: LangGraph's `compile()` does not verify that every state key a node reads is produced upstream. Ordinary compilers have solved the analogous problem for decades—definite-assignment analysis rejects a program that reads a local before it is assigned on some path (Gosling et al., 2014; Kildall, 1973; Aho et al., 2006). `pttai`'s contribution is not that algorithm, which is textbook; it is *applying* it to an LLM pipeline's shared state, and the annotation design (per-node `reads`/`writes`) that makes the shared-state dataflow explicit enough to analyze at build time.

We present `pttai`, a declarative layer over

Figure 1: The > DSL builds a linked node structure in memory; `AgenticGraph` walks it into `add_node`/`add_edge`/`Send` calls; the build-time dataflow validator gates the build (`GraphValidationError` on failure, *before any model call*); on success it compiles to a native LangGraph `StateGraph`.

LangGraph.[1] Our contributions are:

1. **A declarative > DSL over LangGraph** (§2) whose typed per-node `reads`/`writes` annotations make the shared-state dataflow of a pipeline explicit, while compiling down to a native LangGraph `StateGraph` so that streaming, async, checkpointers, and observability are inherited, not reimplemented.
2. **A build-time availability (dataflow) analysis for these graphs** (§2.2): a forward MAY/MUST fixpoint over the graph's acyclic condensation—reaching-definitions / definite-assignment adapted to agent graphs (Kildall, 1973; Aho et al., 2006; Gosling et al., 2014)—that fails the build, with zero model calls, when a node reads a key no upstream node produces, a branch is unwired, or parallel branches write an unreduced key. Because hard errors come only from the MAY (some-path) set, we observe no false positives on our valid corpus.
3. **An evaluation** (§4) whose primary, non-circular arm applies the validator to *LLM-generated* pipelines, supported by a controlled bug suite and a survey of wild bugs.

`pttai` borrows the "ergonomic layer over a powerful runtime" idea from Keras (Chollet et al., 2015), but the ergonomics are not the contribution; the validator is.

2 System Design

2.1 The > DSL and compilation

A `pttai` pipeline is built by instantiating `Node` objects and wiring them with `>` (Figure 1). Wiring is deferred: `a > b` sets `a.child = b` and returns `b`, so `a > b > c` builds a linked structure in memory—no graph edges yet. `AgenticGraph(start, end_nodes)` then walks that structure and emits the actual LangGraph `add_node`/`add_edge`/`add_conditional_edges` calls before compiling. There are three node types: `AgentNode` (a system-prompted model call with an internal tool-call loop), `DecisionNode` (constrained structured-output routing: the model must return one of the declared `choices`, wired `decision["c"] > handler`), and `InputNode` (human-in-the-loop via LangGraph's `interrupt`). Fan-out is `fanout(a, b)` and map-reduce is `.map(field)`.

Nodes read and write a shared, reducer-based state. Each node declares its `reads` and `writes`; a node returns only the keys it updates and the channel reducers merge them, which is what makes parallel branches and checkpointing correct. Those declarations are also what the validator analyzes. Crucially, an `AgenticGraph` *is* a LangGraph `StateGraph` and its compiled artifact is LangGraph's `CompiledStateGraph`: `pttai` adds a build pass on top of LangGraph and then hands the compiled graph to it. Nothing in the runtime is reimplemented.

2.2 The build-time dataflow validator

When `AgenticGraph(...)` is constructed, it runs a static analysis over the wired nodes and **fails the build** on a hard error, raising `GraphValidationError` before `compile()`. This is the contribution.

The analysis. During the build we record a tagged edge list: each edge is `(src, dst, kind)` where `kind` is `seq` (an unconditional, AND edge) or `cond` (an exclusive, OR edge: a `DecisionNode` choice or a `Send` fan-out target). We compute the graph's acyclic condensation by excluding loop-back edges (found by a DFS), then compute, per node, two sets of available state keys by forward fixpoint:

- MAY—keys available on *some* path into the node. A monotone-*increasing* union fixpoint.
- MUST—keys guaranteed on *every* path into the node. A monotone-*decreasing* fixpoint.

The transfer function is the standard forward one—a node's out-set is its in-set plus the keys it writes—with two combining rules at joins that distinguish this from a scalar CFG analysis: *union across an* AND-*parallel join* (a node after `fanout(a,b)` sees both branches' writes), and *intersect across an exclusive* OR-*group* (a node after a `DecisionNode` is guaranteed only the keys every choice branch

[1] Source, examples, benchmark, and demo: https://github.com/TeeratP/agentic-framework.

produces). The seed is the *input* keys—reduced channels, schema keys no node writes, and user-declared inputs—not all declared keys; seeding with inputs rather than the whole schema is what lets the analysis catch a read of a *computed* key before its producer has run. Excluding loop-back edges makes the analysis reflect first-iteration reality: a key produced only by a downstream node that cycles back is *not* credited as available on entry, so loop-carried read-before-write is flagged rather than silently accepted. The fixpoint iterates until a full sweep changes nothing, bounded by the node count.

Errors from MAY, warnings from MUST. Hard *errors* are raised only from the MAY set: a read is an error only when the key is available on *no* path into the node. If a key is available on some but not all paths (MAY but not MUST), that is a *warning*, never a build failure. This is why we observe no false positives: the analysis reports "used before produced" only when *no* path produces the key, exactly as definite-assignment analysis does (Gosling et al., 2014; Nielson et al., 1999). We are careful *not* to claim this as a soundness proof (see Limitations): it is sound for the errors it raises given the declared annotations, not complete, and the 0-false-positive figure is a measurement on our corpus.

What it checks. On top of the dataflow sets, `pttai` raises seven error classes and four warning classes (Table 1). Errors fail the build; warnings surface in a `model.summary()`-style table and can be promoted to errors with `strict=True`.

Error (fails build)	**Condition**
read-before-write	reads a computed key produced only downstream / on a sibling branch (MAY)
undeclared key	reads/writes a key absent from the state schema
concurrent write, no reducer	two parallel branches write the same non-reduced key
dangling choice	a `DecisionNode` choice with no wired handler
dead-end node	a non-terminal node with no child‡
duplicate node name	two distinct nodes share a name†
prompt/read mismatch	a `node_prompt` `{placeholder}` that is not a declared scalar read
Warning (build succeeds)	**Condition**
some-paths read	key in MAY but not MUST
optional read unmet	`reads=["x?"]` never produced
unreachable node	not reachable from the start node
end node w/ children	a declared end node still has a child

Table 1: The validator's error and warning classes. †Duplicate names and dead ends are raised structurally during the build walk (as `ValueError`); the rest come from the dataflow pass. ‡The dead-end rejection is a `pttai` DSL-strictness choice, *not* a LangGraph bug: LangGraph treats a childless node as a legal implicit terminal (§4.2). A dead read (a scalar read never interpolated) is an additional warning.

3 Demonstration

The demonstration is a browser playground (Figure 2). A reviewer edits a small `pttai` snippet—one of the shipped NLP pipelines (RAG QA, extract→summarize, document triage) or their own—and submits it. On a clean build the playground renders the *compiled LangGraph* as a node/edge diagram plus the validator's `summary()` table (every node's reads, writes, and available keys). The intended live moment: take a working RAG-QA pipeline and reorder two nodes so the answer step reads `passages` before the retriever runs. Raw LangGraph would compile this and only fail after the retrieval model call. `pttai` instead rejects it at build: the diagram is redrawn from the pre-compile wiring with the **offending node painted red** and the exact error attached ("`reads computed key 'passages' but no upstream node produces it`"), with *zero model calls made*. The reviewer sees the picture of the bug before running anything.

The playground itself never invokes a model—it builds, validates, and draws. To show that `pttai` *is* a working agent framework and not only a linter, the planned screencast will first run a live pipeline (the parallelization example, which really calls the model) end-to-end, and only then switch to the playground for the build-time catch. Because the playground `exec`s user-supplied code, it is sandboxed for public hosting: the snippet is size-capped, AST-checked (imports allow-listed to `pttai`/`typing`/safe stdlib; dunder access and dangerous builtins such as `eval`/`open`/`__import__` rejected), run with a restricted `__builtins__`, and executed under a per-request timeout. The model backend is an offline fake, so the demo needs no API key and makes no network call. A <2.5-

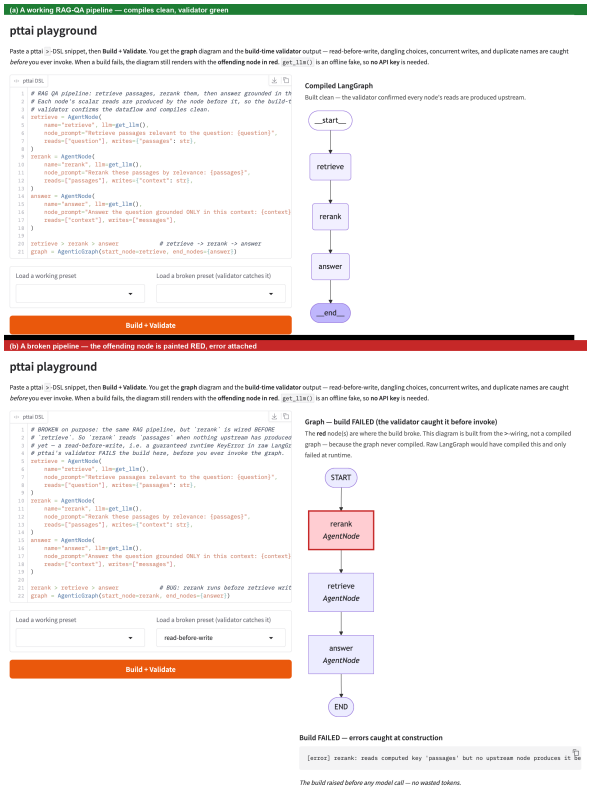

Figure 2: The playground. A broken pipeline is rejected at build time and drawn with the offending node in red and the validator error attached—no model call is made.

minute screencast will accompany the final submission.

4 Evaluation

All evaluations run offline (no API key); the harness is in the repository under `eval/`. The controlled suite and wild-bug survey are authored by us and are therefore vulnerable to the objection that a validator catches only the bugs its author planted. Our *primary* evaluation is designed to remove that objection.

4.1 Primary: LLM-generated pipelines

The most 2026-relevant question is whether the validator is a useful static guardrail for *AI-written* agent code. We therefore designed a study (`eval/llmgen/`) in which a small OpenAI model (`gpt-5.4-nano`) generates `pttai` pipelines from short NLP task specs—given only the docs, and **never told to write bugs**—after which every generated pipeline is built through the validator and every flag is author-adjudicated as a true bug or a false positive per a fixed protocol (`adjudicate.md`). Because the bug distribution comes from a model solving ordinary tasks, whatever the validator catches, misses, or wrongly flags is a fact about real AI-written code, not about a hand-picked corpus. The protocol targets $N \geq 100$ generated pipelines; construction alone scores a pipeline (the validator runs inside `AgenticGraph.__init__`), so scoring needs no model call, only the generation step does. The scoring path is verified offline on a committed 9-pipeline sample set (4 clean, 5 flagged, 0 false positives), but that is a harness self-test, not the study.

> **TODO(author):** populate from `eval/llmgen/run_study.sh`. Report N, the fraction of *buildable* pipelines flagged, the *author-adjudicated* false-positive rate among flags, and the split of flags by the LangGraph phase (runtime / silent / also-at-build). These require an `OPENAI_API_KEY` and a keyed generation run that the build environment cannot perform; **we do not report or claim them here**.

4.2 Controlled bug suite

`bugbench` is a labeled unit-test suite of 17 buggy and 19 valid pipelines that pins the validator's behavior per bug class; it is not the headline result. For each item we record whether `pttai` fails the build and—using an offline fake model—in which phase raw LangGraph would surface the same bug. `pttai` fails the build on **all 17** buggy pipelines with the correct diagnostic (median build time under 1 ms) and **0** false positives on the 19 valid pipelines. We partition the buggy set into three honest categories (Table 2):

- **Differentiator (12 items, 5 classes)**: read-before-write, cyclic-read-before-write, dangling-choice, concurrent-write-no-reducer, prompt-placeholder-mismatch. `pttai` catches all 12 at build; raw LangGraph catches **0** at build and surfaces all **12** at run time.
- **Caught by both (2 items)**: duplicate node names. Both frameworks reject these at build (`ValueError`), so they are *not* a differentiator—we say so explicitly.
- **DSL-strictness (3 items, excluded)**: dead-end nodes. LangGraph 1.2 treats a childless node as a legal implicit terminal that compiles and runs (we verified this empirically); `pttai` rejecting it is a strictness choice of our DSL, *not* a LangGraph bug, so we exclude these from the differentiator claim.

On the differentiator subset, the offline harness attributes **8** LangGraph model calls to reaching the

Category / class	n	pttai	LangGraph
Differentiator (build-catch only in pttai)			
read-before-write	3	build	runtime
cyclic-read-before-write	2	build	runtime
dangling-choice	3	build	runtime
concurrent-write/no-red.	2	build	runtime
prompt/read mismatch	2	build	runtime
subtotal	**12**	**12/12**	**0/12**
Caught by both (not a differentiator)			
duplicate node names	2	build	build
DSL-strictness (excluded)			
dead-end node	3	build	runs

Table 2: bugbench outcomes by category (`eval/bugbench/results.json`). "build" = rejected at construction; "runtime" = raises only at `invoke()`; "runs" = LangGraph executes it as a legal terminal. `pttai` additionally raises 0 false positives on 19 valid pipelines.

Bug class (runtime symptom)	Reports
read-before-write (`KeyError` on a state key)	2
concurrent write, no reducer (`InvalidUpdateError`)	2
dangling conditional edge	2
Total	**6**

Table 3: Independent, publicly reported wild bugs against raw LangGraph, by targeted class—an illustrative sample, not a census (details and issue links in `docs/EVIDENCE.md`).

bug against `pttai`'s **0**. We flag this number as *simulated*: it is produced by a deterministic offline fake model under a worst-case node ordering, not a measurement of real token cost.

4.3 Wild-bug survey

To show the taxonomy is not author-imagined, we mined public issue trackers (Table 3): six independent, verified reports against raw LangGraph, two per class, each a runtime failure that `pttai`'s build-time check would have caught first. This is an *illustrative sample*, not a frequency claim: these are six issues drawn from thousands, with no denominator, so they establish that the classes occur in the wild, not how often. (Duplicate names and the prompt/read check are excluded: the former both frameworks already catch at build, and the latter is a `pttai`-internal reformulation of a generic `str.format` error with no LangGraph-specific report.)

4.4 Conciseness and overhead

As a side effect of folding `add_node`/`add_edge`/structured-output routing/the tool-call loop into the node abstractions, the `pttai` versions of 12 canonical pipelines total 113 lines against 281 for hand-written LangGraph (`eval/loc_results.csv`); this is a consequence of the DSL, not the point of the system. The added build pass has a sub-millisecond to low-single-digit-ms build time on every corpus pipeline (`eval/bugbench/results.json`); because `pttai` compiles to a native `CompiledStateGraph`, run time, model-call count, and output are LangGraph's.

5 Related Work

`pttai` is complementary to, not a replacement for, existing frameworks; it adds a build-time state-key dataflow check and compiles down to LangGraph. **LangGraph** (LangChain, 2024) is the runtime `pttai` targets; its `compile()` runs basic structural checks but does not analyze which state keys are produced before they are read—the gap this work fills. Several frameworks already perform genuine build-time *structural* checks, so we do *not* claim to be the first build-time check of any kind: **pydantic-graph** (PydanticAI) (Pydantic Services Inc., 2024) encodes a node's out-edges in the return-type annotation of its `run` method, so a standard static type checker (mypy/pyright) flags a mis-wired edge before the graph runs— a build-time graph-*structure* check—but its state is a single shared object passed uniformly to every node, with no per-node read/write availability analysis; **AutoGen**'s GraphFlow (Wu et al., 2023) and **LlamaIndex** Workflows (Liu and LlamaIndex, 2024) validate graph structure (entry points, reachability, event-type wiring), and **Haystack** (deepset, 2024) type-checks explicit component connections. What none of these does is a shared-state *dataflow* availability analysis (read-before-write / concurrent-unreduced-write). **DSPy** (Khattab et al., 2024) optimizes prompts against a metric; its control flow is arbitrary Python, so there is no declared graph to check. **LMQL** (Beurer-Kellner et al., 2023) and **Guidance** (Guidance AI, 2023) constrain *within* a single generation rather than across a multi-node graph, and **CrewAI** (CrewAI Inc., 2024) orchestrates via roles with no declared node/edge graph. To our knowledge, among the frameworks we surveyed `pttai` is the only one

that runs a build-time state-key *dataflow* availability check, and the only one that lowers to an established graph runtime rather than shipping its own.

Limitations

We state the analysis's holes as owned limitations. (1) The MAY/MUST analysis is *sound for the errors it raises but not complete*: it is not a soundness proof, and it reasons over *availability*, not types or values—a key produced with the wrong type or an empty value still passes. (2) It analyzes only the *declared* `reads`/`writes` and the graph edges; it does **not** read the code inside `ConditionNode` predicates (arbitrary lambdas), tool bodies, or other custom callables, so a state key touched only inside such a callable is invisible unless the node declares it. (3) After the loop-back-edge fix the analysis catches loop-carried read-before-write on the first iteration, but it makes no termination or runaway-loop guarantee: a graph that loops forever still passes the build. (4) The 0-false-positive figure is *observed on 19 valid pipelines*, not a guarantee; the MUST set can also over-approximate exclusivity where a decision handler merges back through an unconditional edge, which can only *under*-warn, never produce a wrong hard error. (5) Requiring `reads=`/`writes=` declarations means an undeclared but genuinely-produced input can be rejected until it is declared, and the DSL itself has wiring footguns (e.g. `[b,c] > [d,e]`). (6) The wild-bug survey is an illustrative sample, not a census; and the primary LLM-generation study is designed and its scoring path verified, but its headline numbers await a keyed generation run (§4)—we do not report or claim them here.

Ethics and Broader Impact

`pttai`'s goal is to catch broken agent pipelines *before* they run, which directly reduces wasted model calls—and thus the tokens, cost, latency, and energy spent on runs that were doomed at build time—and avoids committing tool side effects on such runs. The public playground `exec`s user-submitted code; we sandbox it (allow-listed imports, restricted builtins, AST rejection of dangerous calls, size and time caps, offline fake model) so that hosting it does not expose the host, but we note that `exec`-based sandboxes are best-effort and a hardened deployment should add OS-level isolation. The validator makes no claim about the *safety, factuality, or bias of model outputs*; it checks pipeline dataflow, not what the models say. `pttai` is released under the MIT license.

References

Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. 2006. *Compilers: Principles, Techniques, and Tools*, 2nd edition. Addison-Wesley.

Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting is programming: A query language for large language models. In *Proceedings of the ACM on Programming Languages (PLDI)*. ArXiv:2212.06094.

François Chollet et al. 2015. Keras. https://keras.io.

CrewAI Inc. 2024. CrewAI: Framework for orchestrating role-playing, autonomous ai agents. https://github.com/crewAIInc/crewAI. Docs: https://docs.crewai.com/. Accessed 2026.

deepset. 2024. Haystack: An open-source framework for building production-ready llm applications. https://github.com/deepset-ai/haystack. Docs: https://docs.haystack.deepset.ai/. Accessed 2026.

James Gosling, Bill Joy, Guy L. Steele, Gilad Bracha, and Alex Buckley. 2014. *The Java Language Specification, Java SE 8 Edition*. Addison-Wesley. Chapter 16, Definite Assignment.

Guidance AI. 2023. Guidance: A guidance language for controlling large language models. https://github.com/guidance-ai/guidance. Accessed 2026.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations (ICLR)*. ArXiv:2310.03714.

Gary A. Kildall. 1973. A unified approach to global program optimization. In *Proceedings of the 1st ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 194–206.

LangChain. 2024. LangGraph: Low-level orchestration framework for stateful, multi-actor agent applications. https://github.com/langchain-ai/langgraph. Documentation: https://langchain-ai.github.io/langgraph/. Accessed 2026.

Jerry Liu and LlamaIndex. 2024. LlamaIndex workflows. https://developers.llamaindex.ai/python/workflows/. Accessed 2026.

Flemming Nielson, Hanne Riis Nielson, and Chris Hankin. 1999. *Principles of Program Analysis*. Springer.

Pydantic Services Inc. 2024. pydantic-graph: Graph and finite-state-machine library (PydanticAI). https://ai.pydantic.dev/graph/. Accessed 2026.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *Preprint*, arXiv:2308.08155. GraphFlow: https://microsoft.github.io/autogen/.

A Reproducing the evaluation

All controlled numbers are produced offline (no API key) from the repository: `eval/bugbench/run.py` regenerates `eval/bugbench/results.json` (the 17 buggy / 19 valid outcomes, per-class categories, and simulated wasted-call counts); `eval/loc_compare.py` regenerates `eval/loc_results.csv` (the 113 vs. 281 line counts over 12 pipelines); `eval/overhead.py` prints the build-time and runtime-parity measurements; and `eval/llmgen/score.py -gen-dir samples` runs the offline sample-set self-test of the primary study. The primary study's headline numbers require an `OPENAI_API_KEY` and `eval/llmgen/run_study.sh`. The wild-bug reports with issue links are in `docs/EVIDENCE.md`.

B Example: a validated NLP pipeline

The extract-then-summarize pipeline. The `extract` node returns typed structured output; the `summarize` node reads those scalar fields and the validator confirms, at build time, that each is produced upstream before it is read.

```
extract = AgentNode(llm=llm,
node_prompt="Extract fields: {ticket}",
reads=["ticket"],
writes={"product":   str,   "severity":
int})
summarize = AgentNode(llm=llm,
node_prompt="severity-{severity}     on
{product}",
reads=["product", "severity"],
writes=["summary"],
output_field="summary")
extract > summarize
AgenticGraph(start_node=extract,
end_nodes={summarize})
```

Swapping the two nodes (`summarize > extract`) makes `summarize` read `product` and `severity` before `extract` produces them; the build fails with a read-before-write error and no model call is made.